

周计划算法 2026-06-22 ~ 2026-06-26

自动生成于 2026-06-22 | 速刷 Hot 100

2026-06-22 周一

新题 (学习)

LC 0021 合并两个有序链表

题目描述：合并两个有序链表

示例： `list1=[1,2,4], list2=[1,3,4] => [1,1,2,3,4,4]`

LeetCode: <https://leetcode.cn/problems/merge-two-sorted-lists/>

算法核心：迭代法：dummy 头节点 + 双指针遍历两链表，取较小值接入结果链

复杂度：时间 $O(n+m)$ ，空间 $O(1)$

标签：链表，递归

代码实现：

```
public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;

    while (list1 != null && list2 != null) {
        if (list1.val <= list2.val) {
            curr.next = list1;
            list1 = list1.next;
        } else {
            curr.next = list2;
            list2 = list2.next;
        }
        curr = curr.next;
    }
    curr.next = (list1 != null) ? list1 : list2;
    return dummy.next;
}
```

LC 0023 合并 K 个升序链表

题目描述：合并 K 个升序链表

示例： `lists=[[1,4,5],[1,3,4],[2,6]] => [1,1,2,3,4,4,5,6]`

LeetCode: <https://leetcode.cn/problems/merge-k-sorted-lists/>

算法核心： 优先队列（小顶堆）：将 K 个头节点入堆，每次弹出最小节点接入结果链，并将其 next 入堆

复杂度： 时间 $O(n \log K)$ ，空间 $O(K)$

标签： 链表，分治，堆

代码实现：

```
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);

    for (ListNode node : lists) {
        if (node != null) pq.offer(node);
    }

    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    while (!pq.isEmpty()) {
        ListNode node = pq.poll();
        curr.next = node;
        curr = curr.next;
        if (node.next != null) pq.offer(node.next);
    }
    return dummy.next;
}
```

 **复习**

LC 0019 删除链表的倒数第 N 个结点 (R3)

题目描述： 删除链表的倒数第 N 个结点

示例： head=[1,2,3,4,5], n=2 => [1,2,3,5]

LeetCode： <https://leetcode.cn/problems/>

算法核心： 快慢指针，fast 先走 n+1 步，slow 停在待删节点前驱

复杂度： 时间 $O(n)$ ，空间 $O(1)$

标签： 链表，双指针

代码实现：

```
public class LC_0019_RemoveNthNodeFromEndOfList {

    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0, head);
        ListNode fast = dummy;
        ListNode slow = dummy;

        for (int i = 0; i <= n; i++) {
            fast = fast.next;
        }

        while (fast != null) {
```

```

        fast = fast.next;
        slow = slow.next;
    }

    slow.next = slow.next.next;
    return dummy.next;
}

public static void main(String[] args) {
    LC_0019_RemoveNthNodeFromEndOfList s = new LC_0019_RemoveNthNodeFromEndOfList();
    System.out.println(java.util.Arrays.toString(ListNode.toArray(
        s.removeNthFromEnd(ListNode.fromArray(new int[]{1, 2, 3, 4, 5}), 2))));
}
}

```

LC 0160 相交链表，找出两链表相交的起始节点 (R2)

题目描述： 相交链表，找出两链表相交的起始节点

示例： listA=[4,1,8,4,5], listB=[5,6,1,8,4,5], 相交于值为 8 的节点

LeetCode: <https://leetcode.cn/problems/>

算法核心： 双指针交替遍历两链，走完后切换头，相遇处为交点或 null

复杂度： 时间 $O(m+n)$, 空间 $O(1)$

标签： 链表, 双指针

代码实现：

```

public class LC_0160_IntersectionOfTwoLinkedLists {

    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        if (headA == null || headB == null) {
            return null;
        }

        ListNode a = headA;
        ListNode b = headB;
        while (a != b) {
            a = a == null ? headB : a.next;
            b = b == null ? headA : b.next;
        }
        return a;
    }

    public static void main(String[] args) {
        LC_0160_IntersectionOfTwoLinkedLists s = new LC_0160_IntersectionOfTwoLinkedLists();
        ListNode common = ListNode.fromArray(new int[]{8, 4, 5});
        ListNode a = new ListNode(4, new ListNode(1, common));
        ListNode b = new ListNode(5, new ListNode(6, new ListNode(1, common)));
        ListNode node = s.getIntersectionNode(a, b);
        System.out.println("Intersection val: " + (node == null ? "null" : node.val));
    }
}

```

新题 (学习)

LC 0148 排序链表

题目描述：排序链表

示例： `head=[4,2,1,3] => [1,2,3,4]`

LeetCode: <https://leetcode.cn/problems/sort-list/>

算法核心：归并排序（自底向上）：快慢指针找中点切分，递归排序后合并，满足 $O(1)$ 空间要求

复杂度：时间 $O(n \log n)$ ，空间 $O(\log n)$

标签：链表，排序，归并

代码实现：

```
public ListNode sortList(ListNode head) {
    if (head == null || head.next == null) return head;

    // 快慢指针找中点
    ListNode slow = head, fast = head.next;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    ListNode mid = slow.next;
    slow.next = null;

    ListNode left = sortList(head);
    ListNode right = sortList(mid);
    return merge(left, right);
}

private ListNode merge(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), curr = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) {
            curr.next = l1; l1 = l1.next;
        } else {
            curr.next = l2; l2 = l2.next;
        }
        curr = curr.next;
    }
    curr.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

LC 0025 K 个一组翻转链表

题目描述：K 个一组翻转链表

示例： head=[1,2,3,4,5], k=2 => [2,1,4,3,5]

LeetCode: <https://leetcode.cn/problems/reverse-nodes-in-k-group/>

算法核心： 分组翻转：dummy 头节点，每次定位 K 个节点的区间，区间内翻转后接回，不足 K 不翻转

复杂度： 时间 O(n)，空间 O(1)

标签： 链表，翻转

代码实现：

```
public ListNode reverseKGroup(ListNode head, int k) {
    ListNode dummy = new ListNode(0, head);
    ListNode prev = dummy;

    while (true) {
        // 定位第 K 个节点
        ListNode end = prev;
        for (int i = 0; i < k && end != null; i++) {
            end = end.next;
        }
        if (end == null) break;

        ListNode start = prev.next;
        ListNode next = end.next;

        // 翻转 [start, end]
        ListNode curr = start, prevNode = next;
        while (curr != next) {
            ListNode tmp = curr.next;
            curr.next = prevNode;
            prevNode = curr;
            curr = tmp;
        }

        prev.next = end;
        prev = start;
    }
    return dummy.next;
}
```

 **复习**

LC 0092 反转链表 II，反转位置 left 到 right 之间的节点 (R4)

题目描述： 反转链表 II，反转位置 left 到 right 之间的节点

示例： head=[1,2,3,4,5], left=2, right=4 => [1,4,3,2,5]

LeetCode: <https://leetcode.cn/problems/>

算法核心： 找到 left 前驱，区间内头插法逐节点移到区间头部

复杂度： 时间 O(n)，空间 O(1)

标签： 链表

代码实现：

```

public class LC_0092_ReverseLinkedListII {

    public ListNode reverseBetween(ListNode head, int left, int right) {
        if (head == null || left == right) {
            return head;
        }

        ListNode dummy = new ListNode(0, head);
        ListNode prev = dummy;
        for (int i = 1; i < left; i++) {
            prev = prev.next;
        }

        ListNode curr = prev.next;
        for (int i = 0; i < right - left; i++) {
            ListNode next = curr.next;
            curr.next = next.next;
            next.next = prev.next;
            prev.next = next;
        }
        return dummy.next;
    }

    public static void main(String[] args) {
        LC_0092_ReverseLinkedListII s = new LC_0092_ReverseLinkedListII();
        System.out.println(java.util.Arrays.toString(ListNode.toArray(
            s.reverseBetween(ListNode.fromArray(new int[]{1, 2, 3, 4, 5}), 2, 4))));
    }
}

```

LC 0142 环形链表 II，返回环入口节点，无环返回 null (R3)

题目描述： 环形链表 II，返回环入口节点，无环返回 null

示例： head=[3,2,0,-4], pos=1 => 返回值为 2 的节点

LeetCode： <https://leetcode.cn/problems/>

算法核心： Floyd 快慢指针判环，相遇后一指针从头走，再相遇处为入口

复杂度： 时间 O(n)，空间 O(1)

标签： 链表，双指针

代码实现：

```

public class LC_0142_LinkedListCycleII {

    public ListNode detectCycle(ListNode head) {
        if (head == null || head.next == null) {
            return null;
        }

        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
    }
}

```

```

        if (slow == fast) {
            ListNode ptr = head;
            while (ptr != slow) {
                ptr = ptr.next;
                slow = slow.next;
            }
            return ptr;
        }
    }
    return null;
}

public static void main(String[] args) {
    LC_0142_LinkedListCycleII s = new LC_0142_LinkedListCycleII();
    ListNode node4 = new ListNode(-4);
    ListNode node3 = new ListNode(0, node4);
    ListNode node2 = new ListNode(2, node3);
    ListNode node1 = new ListNode(3, node2);
    node4.next = node2;
    ListNode entry = s.detectCycle(node1);
    System.out.println("Cycle entry val: " + (entry == null ? "null" : entry.val));
}
}

```

2026-06-24 周三

新题 (学习)

LC 0094 二叉树的中序遍历

题目描述： 二叉树的中序遍历

示例： root=[1,null,2,3] => [1,3,2]

LeetCode： <https://leetcode.cn/problems/binary-tree-inorder-traversal/>

算法核心： 递归：左→根→右；迭代：栈模拟，先深入左子树，出栈访问，进入右子树

复杂度： 时间 O(n)，空间 O(n)

标签： 二叉树，遍历，栈

代码实现：

```

// 递归
public List<Integer> inorderTraversal(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    inorder(root, res);
    return res;
}

private void inorder(TreeNode node, List<Integer> res) {
    if (node == null) return;
    inorder(node.left, res);
    res.add(node.val);
}

```

```

        inorder(node.right, res);
    }

    // 迭代（栈）
    public List<Integer> inorderTraversalIter(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode curr = root;
        while (curr != null || !stack.isEmpty()) {
            while (curr != null) {
                stack.push(curr);
                curr = curr.left;
            }
            curr = stack.pop();
            res.add(curr.val);
            curr = curr.right;
        }
        return res;
    }
}

```

LC 0104 二叉树的最大深度

题目描述： 二叉树的最大深度

示例： root=[3,9,20,null,null,15,7] => 3

LeetCode: <https://leetcode.cn/problems/maximum-depth-of-binary-tree/>

算法核心： 递归：max(左子树深度, 右子树深度) + 1; 层序遍历：BFS 统计层数

复杂度： 时间 O(n), 空间 O(n)

标签： 二叉树, DFS, BFS

代码实现：

```

// 递归
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1;
}

// BFS（层序遍历）
public int maxDepthBFS(TreeNode root) {
    if (root == null) return 0;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    int depth = 0;
    while (!q.isEmpty()) {
        int size = q.size();
        depth++;
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
    }
}

```

```
    return depth;
}
```

复习

LC 0024 两两交换链表中的节点，不能修改节点值 (R4)

题目描述： 两两交换链表中的节点，不能修改节点值

示例： [1,2,3,4] => [2,1,4,3]

LeetCode： <https://leetcode.cn/problems/>

算法核心： dummy 节点，每次交换 prev 后两个节点

复杂度： 时间 $O(n)$ ，空间 $O(1)$

标签： 链表，递归

代码实现：

```
public class LC_0024_SwapNodesInPairs {

    public ListNode swapPairs(ListNode head) {
        ListNode dummy = new ListNode(0, head);
        ListNode prev = dummy;

        while (prev.next != null && prev.next.next != null) {
            ListNode first = prev.next;
            ListNode second = prev.next.next;

            first.next = second.next;
            second.next = first;
            prev.next = second;
            prev = first;
        }
        return dummy.next;
    }

    public static void main(String[] args) {
        LC_0024_SwapNodesInPairs s = new LC_0024_SwapNodesInPairs();
        System.out.println(java.util.Arrays.toString(ListNode.toArray(
            s.swapPairs(ListNode.fromArray(new int[]{1, 2, 3, 4})))));
    }
}
```

LC 0056 合并所有重叠的区间 (R5)

题目描述： 合并所有重叠的区间

示例： intervals=[[1,3],[2,6],[8,10],[15,18]] => [[1,6],[8,10],[15,18]]

LeetCode： <https://leetcode.cn/problems/>

算法核心： 按起点排序，与结果集最后一个区间比较，重叠则合并

复杂度： 时间 $O(n \log n)$ ，空间 $O(n)$

标签: 数组, 排序

代码实现:

```
public class LC_0056_MergeIntervals {  
  
    public int[][] merge(int[][] intervals) {  
        if (intervals == null || intervals.length == 0) return new int[0][];  
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);  
        List<int[]> res = new ArrayList<>();  
        res.add(intervals[0]);  
        for (int i = 1; i < intervals.length; i++) {  
            int[] last = res.get(res.size() - 1);  
            int[] cur = intervals[i];  
            if (cur[0] <= last[1]) {  
                last[1] = Math.max(last[1], cur[1]);  
            } else {  
                res.add(cur);  
            }  
        }  
        return res.toArray(new int[0][]);  
    }  
}
```

2026-06-25 周四

📖 新题 (学习)

LC 0226 翻转二叉树

题目描述: 翻转二叉树

示例: root=[4,2,7,1,3,6,9] => [4,7,2,9,6,3,1]

LeetCode: <https://leetcode.cn/problems/invert-binary-tree/>

算法核心: 递归: 交换左右子树, 递归翻转左右子节点; 也可用 BFS 层序遍历交换

复杂度: 时间 $O(n)$, 空间 $O(n)$

标签: 二叉树, DFS

代码实现:

```
// 递归  
public TreeNode invertTree(TreeNode root) {  
    if (root == null) return null;  
    TreeNode left = invertTree(root.right);  
    TreeNode right = invertTree(root.left);  
    root.left = left;  
    root.right = right;  
}
```

```
    return root;
}
```

LC 0101 对称二叉树

题目描述： 对称二叉树

示例： root=[1,2,2,3,4,4,3] => true

LeetCode: <https://leetcode.cn/problems/symmetric-tree/>

算法核心： 递归：判断左右子树是否镜像对称（左.left vs 右.right && 左.right vs 右.left）

复杂度： 时间 O(n)，空间 O(n)

标签： 二叉树，DFS，BFS

代码实现：

```
// 递归
public boolean isSymmetric(TreeNode root) {
    if (root == null) return true;
    return isMirror(root.left, root.right);
}
private boolean isMirror(TreeNode t1, TreeNode t2) {
    if (t1 == null && t2 == null) return true;
    if (t1 == null || t2 == null) return false;
    return t1.val == t2.val
        && isMirror(t1.left, t2.right)
        && isMirror(t1.right, t2.left);
}
```

复习

LC 0019 删除链表的倒数第 N 个结点 (R4)

题目描述： 删除链表的倒数第 N 个结点

示例： head=[1,2,3,4,5], n=2 => [1,2,3,5]

LeetCode: <https://leetcode.cn/problems/>

算法核心： 快慢指针，fast 先走 n+1 步，slow 停在待删节点前驱

复杂度： 时间 O(n)，空间 O(1)

标签： 链表，双指针

代码实现：

```
public class LC_0019_RemoveNthNodeFromEndOfList {

    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0, head);
        ListNode fast = dummy;
        ListNode slow = dummy;
```

```

        for (int i = 0; i <= n; i++) {
            fast = fast.next;
        }

        while (fast != null) {
            fast = fast.next;
            slow = slow.next;
        }

        slow.next = slow.next.next;
        return dummy.next;
    }

    public static void main(String[] args) {
        LC_0019_RemoveNthNodeFromEndOfList s = new LC_0019_RemoveNthNodeFromEndOfList();
        System.out.println(java.util.Arrays.toString(ListNode.toArray(
            s.removeNthFromEnd(ListNode.fromArray(new int[]{1, 2, 3, 4, 5}), 2))));
    }
}

```

LC 0002 两数相加，两链表表示非负整数（逆序），返回相同形式的结果链表 (R3)

题目描述： 两数相加，两链表表示非负整数（逆序），返回相同形式的结果链表

示例： l1=[2,4,3], l2=[5,6,4] => [7,0,8] 即 342+465=807

LeetCode: <https://leetcode.cn/problems/>

算法核心： 逐位相加维护 carry, dummy 头节点串结果

复杂度： 时间 $O(\max(m,n))$, 空间 $O(\max(m,n))$

标签： 链表, 数学

代码实现：

```

public class LC_0002_AddTwoNumbers {

    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        int carry = 0;

        while (l1 != null || l2 != null || carry != 0) {
            int sum = carry;
            if (l1 != null) {
                sum += l1.val;
                l1 = l1.next;
            }
            if (l2 != null) {
                sum += l2.val;
                l2 = l2.next;
            }
            carry = sum / 10;
            curr.next = new ListNode(sum % 10);
            curr = curr.next;
        }
        return dummy.next;
    }
}

```

```
public static void main(String[] args) {
    LC_0002_AddTwoNumbers s = new LC_0002_AddTwoNumbers();
    System.out.println(java.util.Arrays.toString(ListNode.toArray(
        s.addTwoNumbers(
            ListNode.fromArray(new int[]{2, 4, 3}),
            ListNode.fromArray(new int[]{5, 6, 4})))));
}
```

2026-06-26 周五

新题 (学习)

LC 0102 二叉树的层序遍历

题目描述： 二叉树的层序遍历

示例： root=[3,9,20,null,null,15,7] => [[3],[9,20],[15,7]]

LeetCode: <https://leetcode.cn/problems/binary-tree-level-order-traversal/>

算法核心： BFS (队列)： 每轮处理当前层所有节点，收集值并添加下一层节点

复杂度： 时间 $O(n)$ ， 空间 $O(n)$

标签： 二叉树， BFS， 队列

代码实现：

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        res.add(level);
    }
    return res;
}
```

LC 0105 从前序与中序遍历序列构造二叉树

题目描述： 从前序与中序遍历序列构造二叉树

示例： preorder=[3,9,20,15,7], inorder=[9,3,15,20,7] => 二叉树 [3,9,20,null,null,15,7]

LeetCode: <https://leetcode.cn/problems/construct-binary-tree-from-preorder-and-inorder-traversal/>

算法核心： 前序第一个为根，在中序中找到根的位置，左半为左子树，右半为右子树，递归构建

复杂度： 时间 $O(n)$, 空间 $O(n)$

标签： 二叉树, 分治, 哈希表

代码实现：

```
public TreeNode buildTree(int[] preorder, int[] inorder) {
    Map<Integer, Integer> idxMap = new HashMap<>();
    for (int i = 0; i < inorder.length; i++) {
        idxMap.put(inorder[i], i);
    }
    return build(preorder, 0, preorder.length - 1,
                inorder, 0, inorder.length - 1, idxMap);
}

private TreeNode build(int[] pre, int pL, int pR,
                       int[] in, int iL, int iR,
                       Map<Integer, Integer> map) {
    if (pL > pR || iL > iR) return null;

    int rootVal = pre[pL];
    TreeNode root = new TreeNode(rootVal);
    int rootIdx = map.get(rootVal);
    int leftSize = rootIdx - iL;

    root.left = build(pre, pL + 1, pL + leftSize,
                      in, iL, rootIdx - 1, map);
    root.right = build(pre, pL + leftSize + 1, pR,
                       in, rootIdx + 1, iR, map);
    return root;
}
```

 **复习**

LC 0142 环形链表 II, 返回环入口节点, 无环返回 null (R4)

题目描述： 环形链表 II, 返回环入口节点, 无环返回 null

示例： head=[3,2,0,-4], pos=1 => 返回值为 2 的节点

LeetCode: <https://leetcode.cn/problems/>

算法核心： Floyd 快慢指针判环, 相遇后一指针从头走, 再相遇处为入口

复杂度： 时间 $O(n)$, 空间 $O(1)$

标签： 链表, 双指针

代码实现：

```

public class LC_0142_LinkedListCycleII {

    public ListNode detectCycle(ListNode head) {
        if (head == null || head.next == null) {
            return null;
        }

        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                ListNode ptr = head;
                while (ptr != slow) {
                    ptr = ptr.next;
                    slow = slow.next;
                }
                return ptr;
            }
        }
        return null;
    }

    public static void main(String[] args) {
        LC_0142_LinkedListCycleII s = new LC_0142_LinkedListCycleII();
        ListNode node4 = new ListNode(-4);
        ListNode node3 = new ListNode(0, node4);
        ListNode node2 = new ListNode(2, node3);
        ListNode node1 = new ListNode(3, node2);
        node4.next = node2;
        ListNode entry = s.detectCycle(node1);
        System.out.println("Cycle entry val: " + (entry == null ? "null" : entry.val));
    }
}

```

LC 0206 反转链表，将单链表原地反转 (R4)

题目描述： 反转链表，将单链表原地反转

示例： [1,2,3,4,5] => [5,4,3,2,1]

LeetCode： <https://leetcode.cn/problems/>

算法核心： 迭代三指针 prev/curr/next，逐个反转 next 指向

复杂度： 时间 $O(n)$ ，空间 $O(1)$

标签： 链表，双指针

代码实现：

```

public class LC_0206_ReverseLinkedList {

    public ListNode reverseList(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;
        while (curr != null) {

```

```
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

public static void main(String[] args) {
    LC_0206_ReverseLinkedList s = new LC_0206_ReverseLinkedList();
    System.out.println(java.util.Arrays.toString(ListNode.toArray(
        s.reverseList(ListNode.fromArray(new int[]{1, 2, 3, 4, 5})))));
}
}
```

 本周算法汇总

学习节奏： 每日 2 道新题 + 2 道复盘

新学算法 (10道)

题号	题目	难度	计划日期
LC_0021	合并两个有序链表	Easy	2026-06-22 周一
LC_0023	合并 K 个升序链表	Hard	2026-06-22 周一
LC_0148	排序链表	Medium	2026-06-23 周二
LC_0025	K 个一组翻转链表	Hard	2026-06-23 周二
LC_0094	二叉树的中序遍历	Easy	2026-06-24 周三
LC_0104	二叉树的最大深度	Easy	2026-06-24 周三
LC_0226	翻转二叉树	Easy	2026-06-25 周四
LC_0101	对称二叉树	Easy	2026-06-25 周四
LC_0102	二叉树的层序遍历	Medium	2026-06-26 周五
LC_0105	从前序与中序遍历序列构造二叉树	Medium	2026-06-26 周五

复习算法 (10道)

题号	题目	轮次	计划日期
LC_0019	删除链表的倒数第 N 个结点	R3	2026-06-22 周一

题号	题目	轮次	计划日期
LC_0160	相交链表，找出两链表相交的起始节点	R2	2026-06-22 周一
LC_0092	反转链表 II，反转位置 left 到 right 之间的节点	R4	2026-06-23 周二
LC_0142	环形链表 II，返回环入口节点，无环返回 null	R3	2026-06-23 周二
LC_0024	两两交换链表中的节点，不能修改节点值	R4	2026-06-24 周三
LC_0056	合并所有重叠的区间	R5	2026-06-24 周三
LC_0019	删除链表的倒数第 N 个结点	R4	2026-06-25 周四
LC_0002	两数相加，两链表表示非负整数（逆序），返回相同形式的结果链表	R3	2026-06-25 周四
LC_0142	环形链表 II，返回环入口节点，无环返回 null	R4	2026-06-26 周五
LC_0206	反转链表，将单链表原地反转	R4	2026-06-26 周五

进度一览

维度	数值
 周期	2026-06-22 ~ 2026-06-26
 新题总量	10 道
 复习总量	10 道
 每日节奏	2 新 + 2 复
 本周涉及题型	P2 链表收尾 → P3 二叉树起步

本周围绕题型

- **P3 二叉树入门**: LC_0021、LC_0094、LC_0104、LC_0226、LC_0101
- **P2 链表收尾**: LC_0023、LC_0025
- **P3 二叉树**: LC_0148、LC_0102、LC_0105

 **建议**: 早地铁学习新题，晚地铁复盘，每周末回顾本周所有题目

由 `scripts/generate-detailed-weekly.mjs` 自动生成